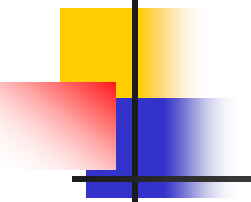


Operating Systems

UNIT-4

Memory Management



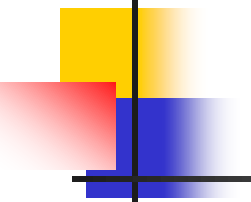
Syllabus

- ❑ Memory Management Strategies:
 - Contiguous Memory Allocation
 - Paging
 - Structure of the Page Table
 - Swapping
- ❑ Virtual Memory Management
 - Demand Paging
 - Page Replacement
 - Allocation of frames
 - Thrashing
- ❑ Storage Management:
 - Overview of Mass Storage Structure
 - HDD Scheduling.



Why Memory Management?

- Programs need memory to run
- Prevents conflicts between processes
- Ensures efficient use of RAM
- Supports multitasking

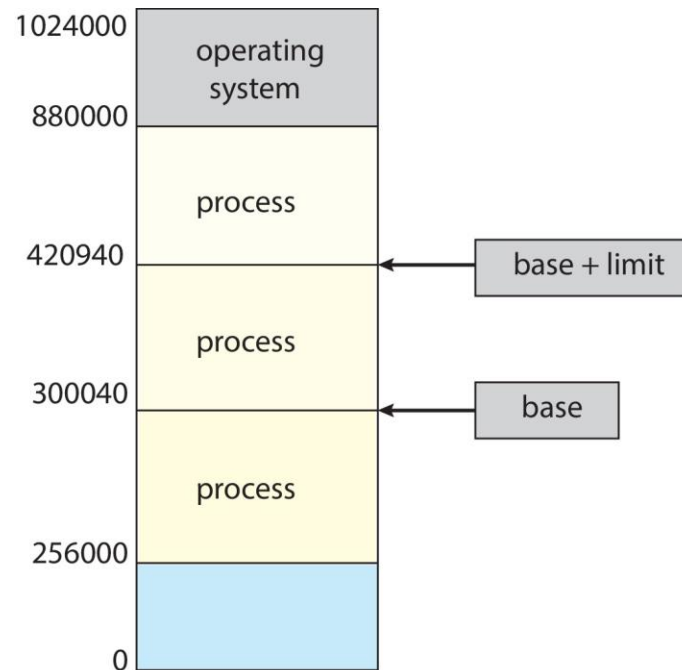


Background

- Program must be brought into memory and placed within a process for it to be run.
- To improve both the **utilization of the CPU** and the **speed** of its response to users, a general-purpose computer must keep several processes in memory.
- OS provides many **Memory Management** schemes to improve memory utilization.
- **Input queue or job queue**, collection of processes on the disk that are waiting to be brought into memory to run the program.
- OS has to perform various steps in order to execute the processes such as address and data binding.

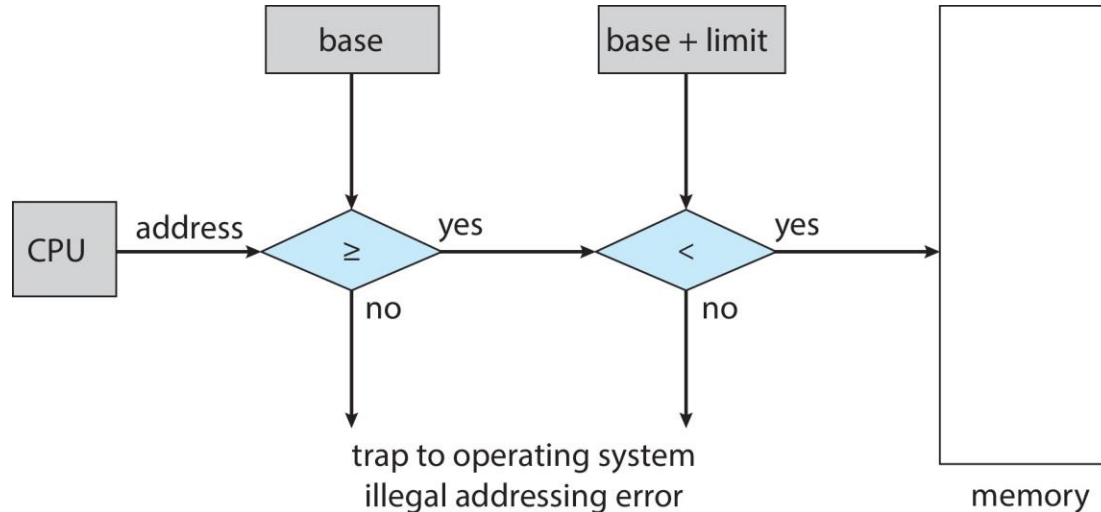
Protection

- Need to ensure that a process can access only those addresses in its address space.
- We can provide this protection by using a pair of **base** and **limit registers** define the logical address space of a process



Hardware Address Protection

- CPU must check every memory access generated in user mode to be sure it is between base and limit for that user.





Address Binding

- There are two types of addresses: logical address and physical address.
- **Logical address** is generated by CPU during a program execution. It is also called as **virtual address**.
- **Physical Address** identifies a physical location in a memory unit.
- Set of all logical addresses generated by CPU in reference to a program is referred as **Logical Address Space**.
- Set of all physical addresses mapped to the corresponding logical addresses is referred as **Physical Address Space**.
- Logical addresses are visible to programs/users.
- Physical addresses are hidden, used internally by hardware.



Address Binding

- **Address binding** is the process of mapping from one address space to another address space.

Address Binding can happen at three different stages:

1. **Compile-time Address Binding:**

- It will be done before loading the program into memory.
- Compiler is responsible for performing address binding.
- Compiler generates **absolute addresses**.
- The compiler requires interacts with an OS memory manager to perform compile-time address binding.
- Requires program to always run in the same memory location.



2. Load time Address Binding:

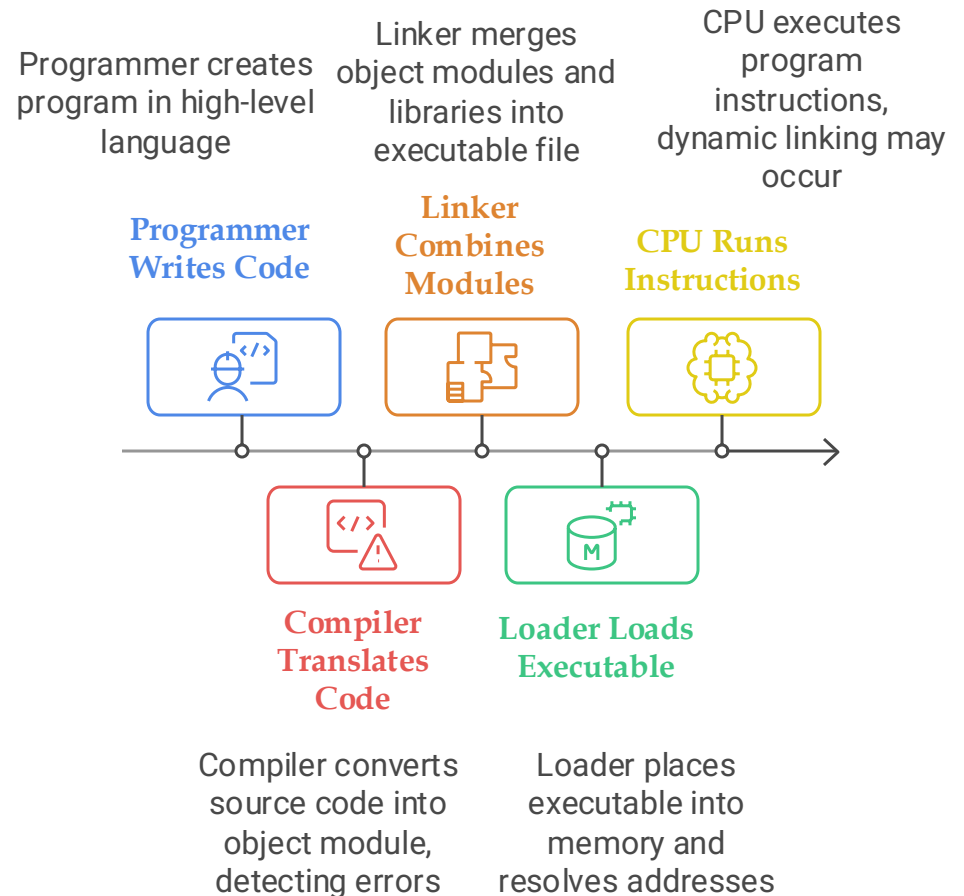
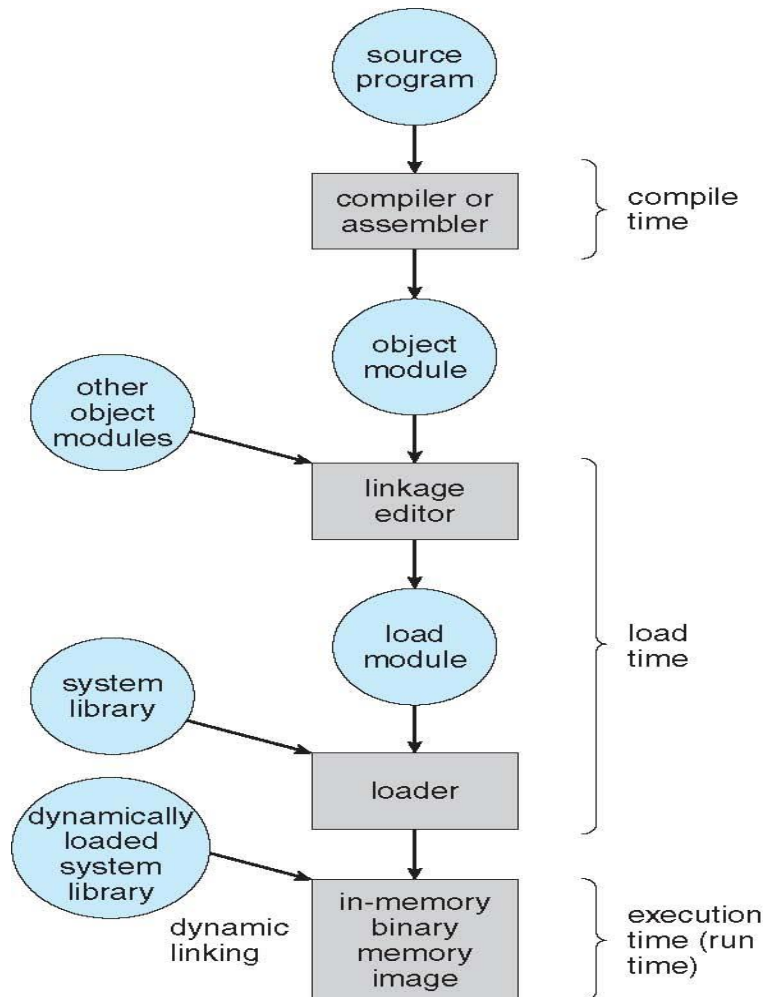
- It will be done after loading the program into memory.
- Compiler generates **relocatable code**.
- Binding is done by the loader when the program is loaded into memory.
- Allows the program to be loaded into different memory locations.

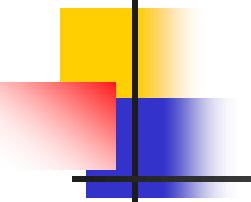
3. Execution time / Dynamic Address Binding:

- Binding done by the processor at the time of program execution.
- Need **hardware support** for address maps.
- The program will be kept on changing the locations in memory until the time of program execution.
- Supports multitasking and virtual memory.

Multistep Processing of a User Program

Program Execution Lifecycle



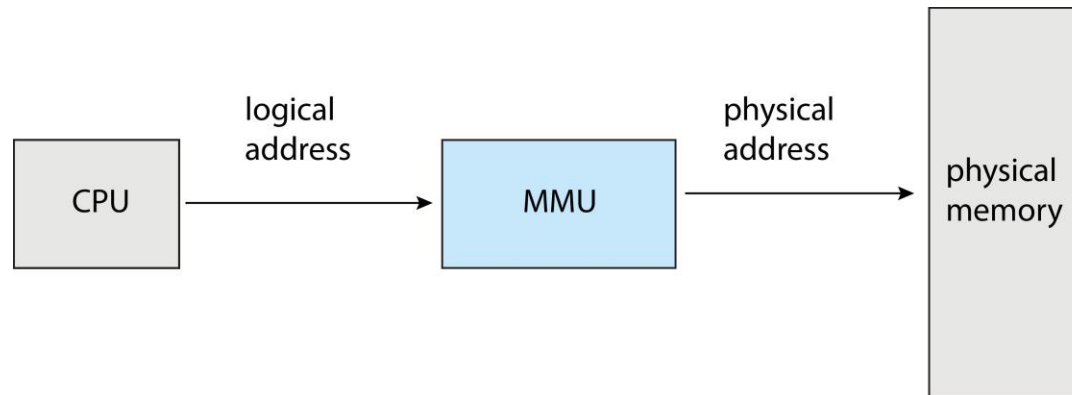


Dynamic Loading

- In dynamic loading, a routine is not loaded until it is called.
- All routines are kept on disk in a relocatable load format.
- The main program is loaded into memory and is executed.
- When a routine needs to call another routine, the calling routine first checks to see whether the other routine has been loaded.
- If it has not, the relocatable linking loader is called to load the desired routine into memory and to update the program's address tables to reflect this change.
- Then control is passed to the newly loaded routine.
- The advantage of dynamic loading is to provide better memory-space utilization.

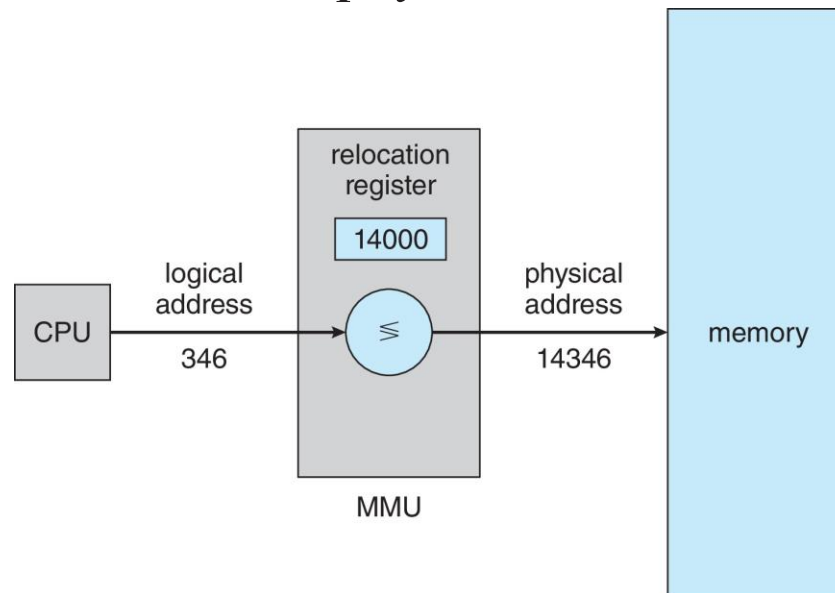
MMU(Memory Management Unit)

- The run time mapping between Virtual address and Physical Address is done by hardware device known as MMU.
- In memory management, Operating System will handle the processes and moves the processes between disk and memory for execution.
- It keeps the track of available and used memory.



MMU(Memory Management Unit)

- The base register now called **relocation register**
- The value in the relocation register is added to every address generated by a user process at the time it is sent to memory
- The user program deals with logical addresses; it never sees the real physical addresses
- Logical address bound to physical addresses



Physical Address = Logical Address + Relocation Register Value



Overlays in Memory Management

- Overlay memory management technique allows multiple programs to be loaded into memory simultaneously, but only a portion of each program is resident in memory at any given time.
- The technique swaps different parts of the programs in and out of memory as required.
- It is commonly used in situations where the memory requirements of the programs exceed the available physical memory.



Advantages

1. Improved Memory Utilization
2. Reduced Memory Fragmentation
3. Reduced Memory Access Time
4. Improved Performance

Disadvantages

1. Increased Complexity
2. Limited Address Space
3. Increased Overhead



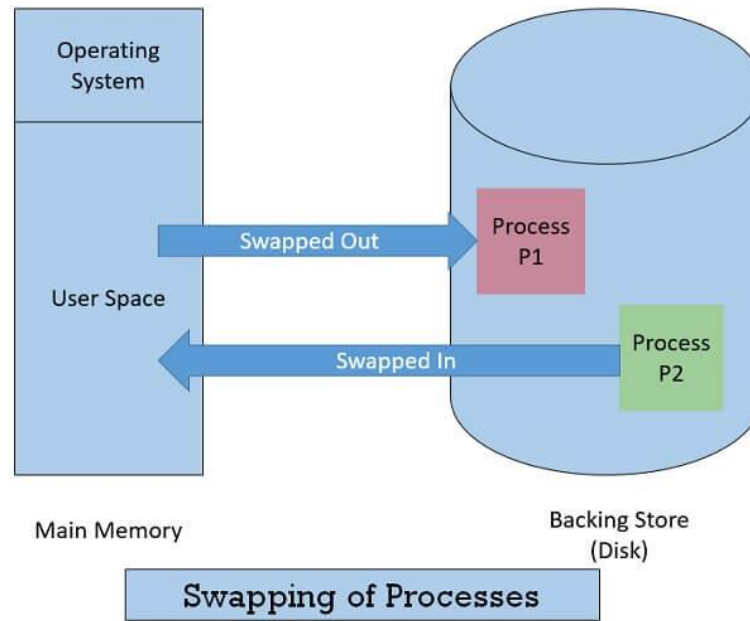
Swapping

- Swapping is a mechanism in which a process can be swapped temporarily out of main memory (or move) to secondary storage (disk) and make that memory available to other processes.
- A process will be moved to temporarily out of memory to a **backing store** and then brought back into memory for continued execution.
- The backing store is a fast disk which must be large enough to accommodate copies of all memory images for all users.
- Swapping makes it possible for the total physical address space of all processes to exceed the real physical memory of the system, thus increasing the **degree of multiprogramming** in a system.

Swapping Process:

1. Select a process to swap out (usually based on priority or idle time).
2. Move its entire memory image to disk (swap space).
3. Load the required process into freed memory.
4. When the swapped-out process is ready to run again, move it back into RAM.





Swap In:

- The method of removing a process from **secondary memory (Hard Drive)** and **restoring it to the main memory (RAM)** for execution is known as the Swap In method.



Swap Out:

- It is a method of bringing out a process from the **main memory(RAM)** and **sending** it to the **secondary memory(hard drive)** so that the processes with higher priority or more memory consumption will be executed known as the Swap Out method.
- **Note:-** Swap In and Swap Out method is done by Medium Term Scheduler(MTS).
- Major part of swap time is transfer time; total transfer time is directly proportional to the amount of memory swapped.

Benefits of Swapping:

- It offers a **higher degree of multiprogramming**.
- Allows **dynamic relocation**.
- It helps to get better **utilization of memory**



Key Concepts:

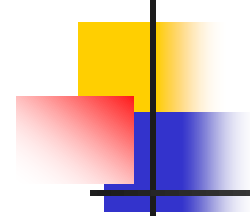
Swap Space: Reserved area on disk for swapped-out processes.

Context Switch Overhead: Swapping involves I/O operations, which are slower than CPU or RAM access.

Thrashing: Excessive swapping occurs if too many processes are in memory, reducing performance.

Benefits of Swapping:

- It offers a **higher degree of multiprogramming**.
- Allows **dynamic relocation**.
- It helps to get better **utilization of memory**



Memory Allocation Techniques

Contiguous Allocation:

Allocate a single contiguous block of memory to a process.

- **Fixed Partitioning:** Memory divided into fixed blocks.
 - Simple, but may waste memory (internal fragmentation).
- **Dynamic Partitioning:** Blocks vary in size according to process needs.
 - Reduces internal fragmentation but may cause external fragmentation.

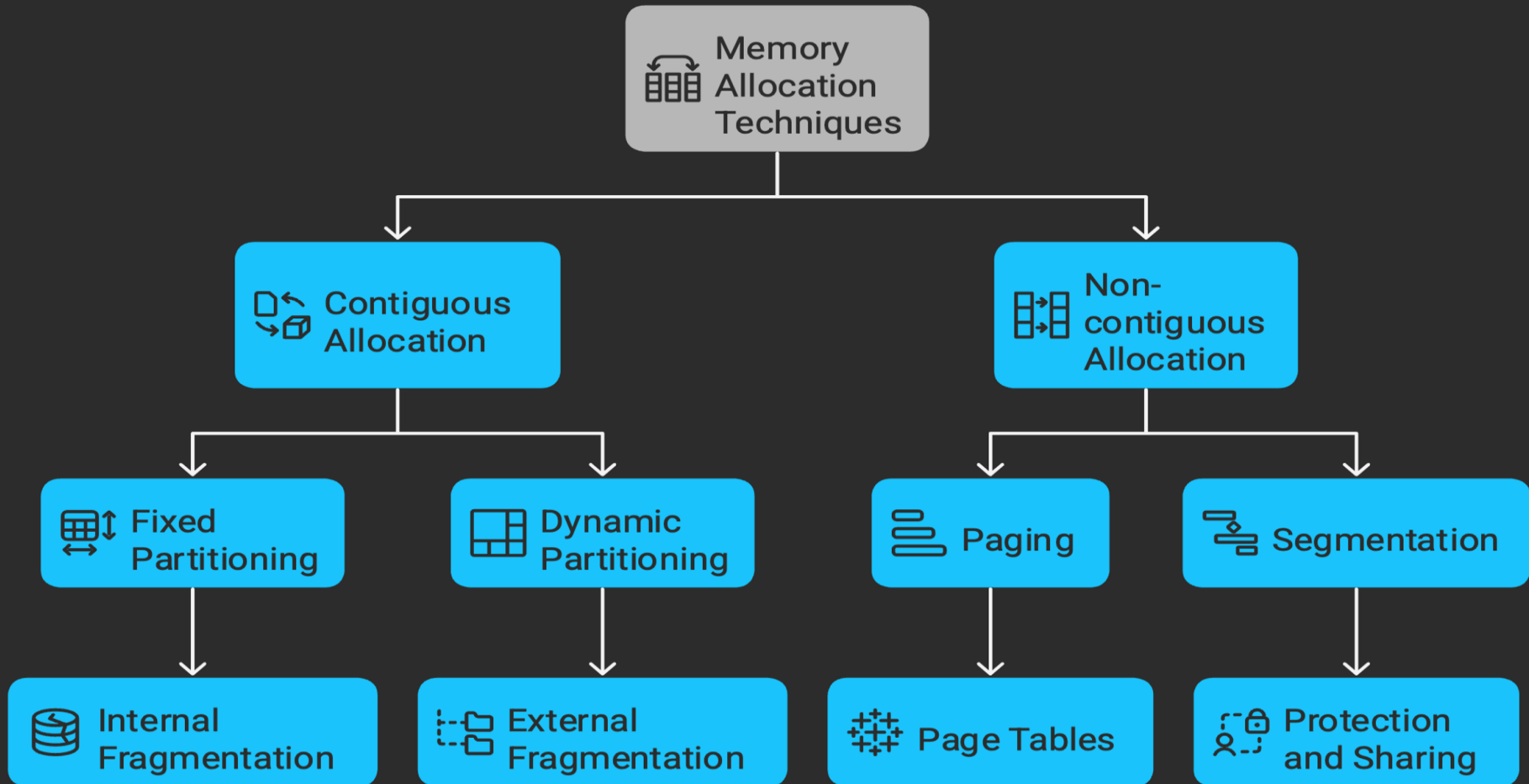
Non-contiguous Allocation:

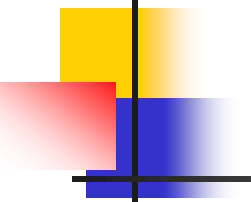
Memory can be allocated in multiple blocks.

- **Paging:**
 - Break memory and processes into fixed-size pages.
 - Eliminates external fragmentation.
 - Uses page tables for address translation.
- **Segmentation:**
 - Memory divided by logical segments (code, data, stack).
 - Supports protection and sharing.

Memory Allocation Techniques

Memory Allocation Techniques





Contiguous Memory Allocation

- The main memory must accommodate both the operating system and the various user processes.
- Therefore we need to allocate main memory in the most efficient way possible.
- The memory is usually divided into two partitions: one for the resident operating system and one for the user processes.
- In contiguous memory allocation, each process is contained in a single section of memory that is contiguous to the section containing the next process.
- Simplest methods for allocating memory is to divide memory into several fixed-sized partitions.
- Each partition may contain exactly one process. Thus, the degree of multiprogramming depends on number of partitions.



Contiguous memory management schemes:

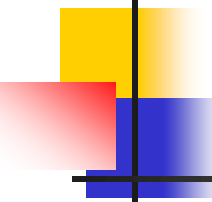
➤ **Single Partition**

In this scheme, the main memory is divided into two contiguous areas or partitions. The operating systems reside permanently in one partition, generally at the lower memory, and the user process is loaded into the other partition.

➤ **Multiple Partition**

The operating system divides the available main memory into multiple parts to load multiple processes into the main memory.

- ✓ **Fixed Partitioning:** Each partition or block is of fixed size and can hold single process.
- ✓ **Dynamic Partitioning:** In this scheme the partitions used are of variable size. Each process occupies only as much memory as they require when loaded for processing.



Fixed Partitioning

Mechanism:

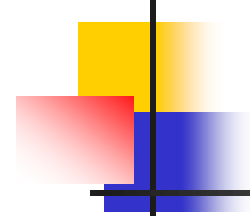
- Each partition has a base address and a limit (size).
- A process can only be loaded if it fits entirely into a partition.

Advantages:

- Simple implementation: Easy to manage and calculate addresses.
- Fast allocation: No need to search for fitting space.

Disadvantages:

- Internal fragmentation: If process size $<$ partition size, memory is wasted.
- Limited flexibility: Large processes may not fit, small ones leave unused space.
- Poor memory utilization when process sizes vary.



Dynamic Partitioning

Mechanism:

- System searches for a suitable free block (first fit, best fit, or worst fit).
- The process is loaded into the selected block.
- **Hole:** block of available memory; holes of various size are scattered throughout memory

Advantages:

- Efficient memory usage: Minimizes wasted space inside partitions.
- Flexible: Can accommodate processes of varying sizes.

Disadvantages:

- External fragmentation: Free memory scattered in small pieces over time.
- Slower allocation: Need to search for a suitable free block.
- Compaction may be needed to reduce fragmentation, adding overhead.



Memory Allocation Methods

A process can be allocated with a free memory partition or block based three strategies:

1. **First fit:**

This type will allocate the first free block that is big enough to satisfy the memory request of a process from the beginning of the memory.

2. **Best fit:**

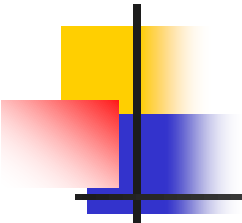
This type will allocate the smallest(minimum) free block that is big enough to satisfy the memory request of a process by searching the whole memory.

3. **Worst fit:**

This will allocates the process to the partition, which is the largest sufficient freely available partition in the main memory.



S.NO.	Contiguous Memory Allocation	Non-contiguous Memory Allocation
1	Contiguous memory allocation allocates consecutive blocks of memory to a file/process.	Non-Contiguous memory allocation allocates separate blocks of memory to a file/process.
2	Faster in Execution.	Slower in Execution.
3	It is easier for the OS to control.	It is difficult for the OS to control.
4	Overhead is minimum as not much address translations are there while executing a process.	More Overheads are there as there are more address translations.
5	Internal fragmentation occurs in Contiguous memory allocation method.	External fragmentation occurs in Non-Contiguous memory allocation method.



6	It includes single partition allocation and multi-partition allocation.	It includes paging and segmentation.
7	Wastage of memory is there.	No memory wastage is there.
8	In contiguous memory allocation, swapped-in processes are arranged in the originally allocated space.	In non-contiguous memory allocation, swapped-in processes can be arranged in any place in the memory.

Compaction: It is process of grouping or combining all the fragmented memory blocks into a single large free block.

Fragmentation

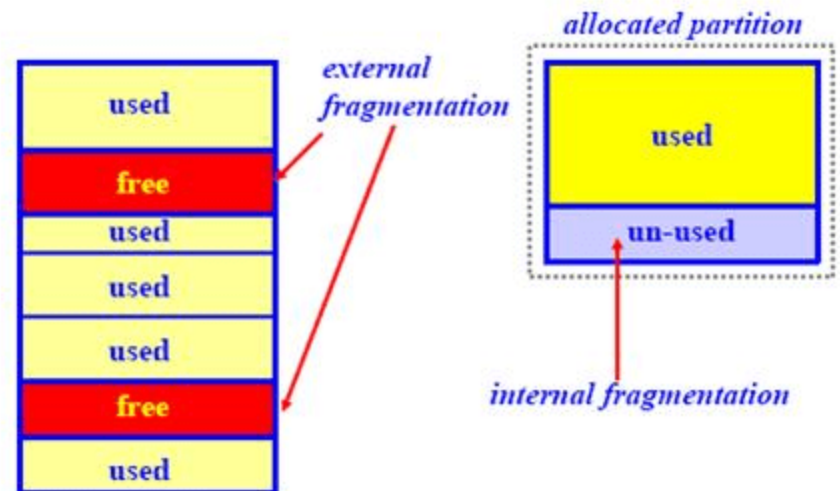
Fragmentation is an unwanted memory space problem in the operating system occurred when the processes are loaded and unloaded from memory.

➤ Internal Fragmentation

When a process with smaller amount of memory request is allocated with a larger memory block, then a free space is created in the given memory block that becomes unused. This condition is called as internal fragmentation.

➤ External Fragmentation

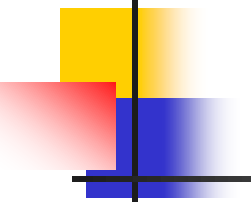
External fragmentation occurs when there is enough memory space to complete a memory request of some process, but it is not contiguous.





S.NO	Internal Fragmentation	External Fragmentation
1	It occurs when free space within a memory block becomes unused.	It occurs when non contiguous free blocks becomes unused due to dynamic allocation.
2	It happens when the method or process is larger than the memory.	It happens when the method or process is removed from memory.
3	The solution of internal fragmentation is best-fit block.	Solution of external fragmentation is compaction, paging and segmentation.
4	Internal fragmentation occurs when memory is divided into fixed sized partitions.	External fragmentation occurs when memory is divided into variable size partitions based on the size of processes.





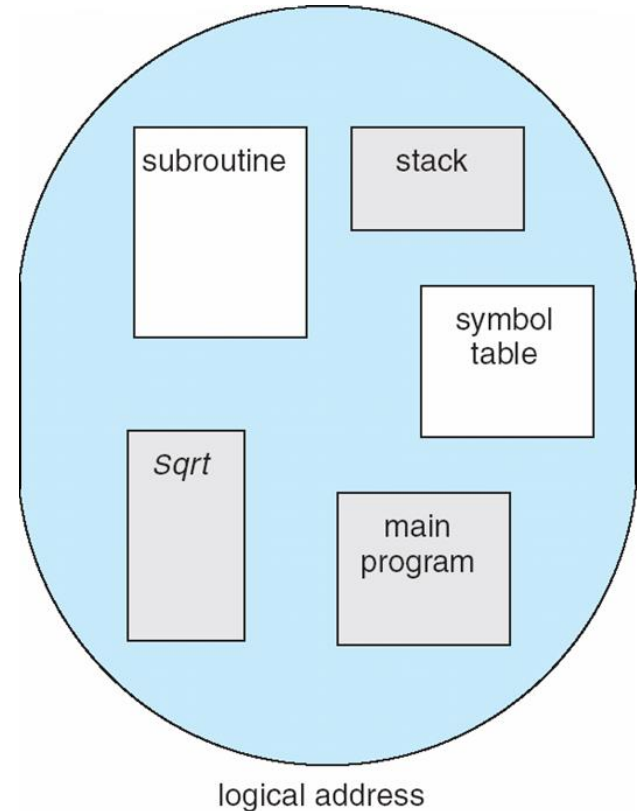
Segmentation

- **Segmentation** is a memory-management scheme that supports programmer view of memory and the logical address space is divided into collection of segments.
- Segmentation permits the physical address space of a process to be noncontiguous.
- Each segment has a name and a length. The addresses specify both the segment name and the offset within the segment.
- For simple implementation segments are numbered and are referred to by a segment number, rather than by a segment name.
- Thus, a logical address consists of a two tuple:
 $\langle \text{segment-number}, \text{offset} \rangle$.



Compiler might create separate segments for the following:

1. The code
2. Global variables
3. The heap, from which memory is allocated
4. The stacks used by each thread
5. The standard C library



User's View of a Program



Segmentation Hardware:

- It is used to map two-dimensional user-defined addresses into one-dimensional physical address with the help of segment table.
- Each entry in the segment table has a segment base and a segment limit.
- The segment base contains the starting physical address where the segment resides in memory, and the segment limit specifies the length of the segment.
- Address generated by the CPU is divided into:
 1. Segment number (s): Number of bits required to represent the segment.
 2. Segment offset (d): Number of bits required to represent the size of the segment.

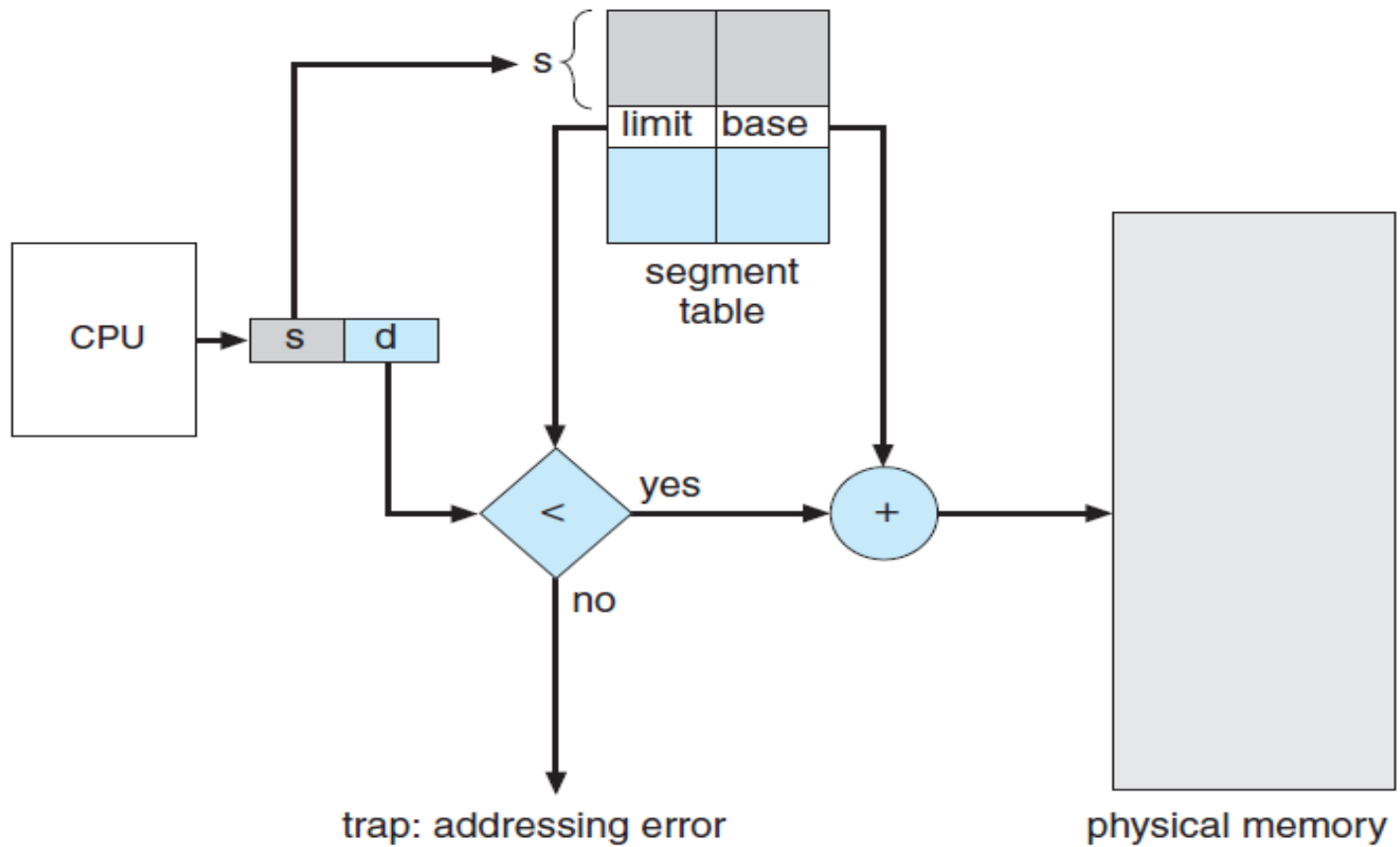


Figure 8.8 Segmentation hardware.

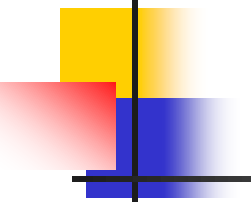


Advantages of Segmentation –

- No Internal fragmentation.
- Segment Table consumes less space in comparison to Page table in paging.

Disadvantage of Segmentation –

- As processes are loaded and removed from the memory, the free memory space is broken into little pieces, causing External fragmentation.



Paging

- **Paging** is another memory-management scheme that also permits **non-contiguous** physical address space.
- In this the main memory is divided into equal size partitions.
- The partitions of the secondary memory are called **pages** while the partitions of the main memory are called **frames**.
- They are divided into equal size partitions to have maximum utilization of the main memory and hence **avoid external fragmentation**.

1. Page size is fixed; same for all processes.

2. Frame size is fixed.

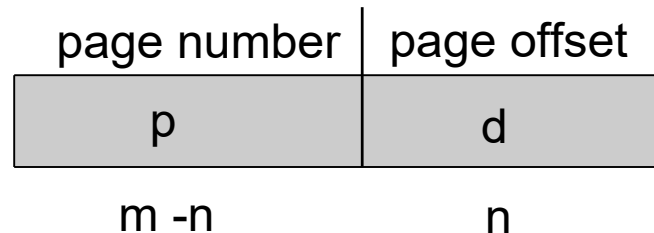
3. Page size and Frame size must be same.



➤ Translation of logical Address into physical Address

The logical address consists of two parts page number and page offset.

1. **Page Number(p)**: It tells the exact page of the process which the CPU wants to access.
2. **Page Offset(d)**: It tells the exact word on that page which the CPU wants to read. (Also called displacement)





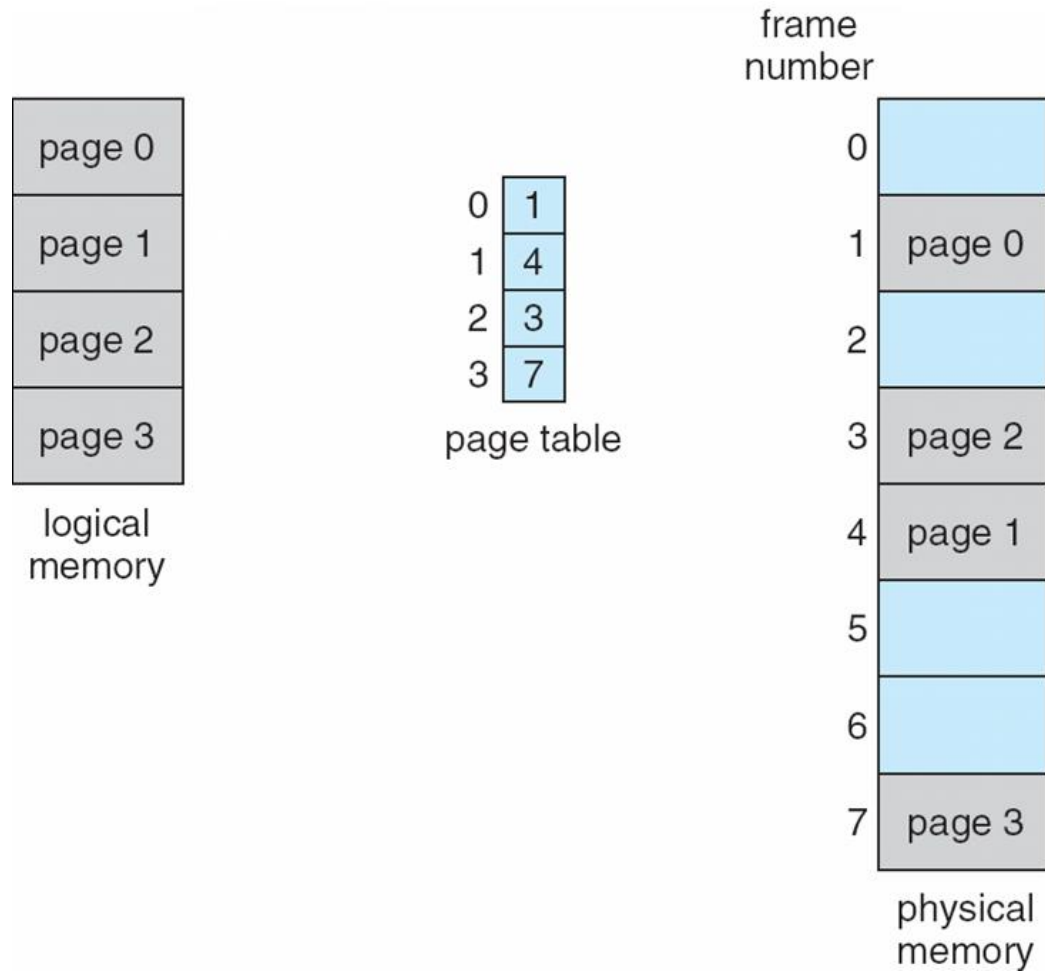
Physical Address: The physical address consists of two parts frame number and page offset.

1. Frame Number(f): It tells the exact frame where the page is stored in physical memory.

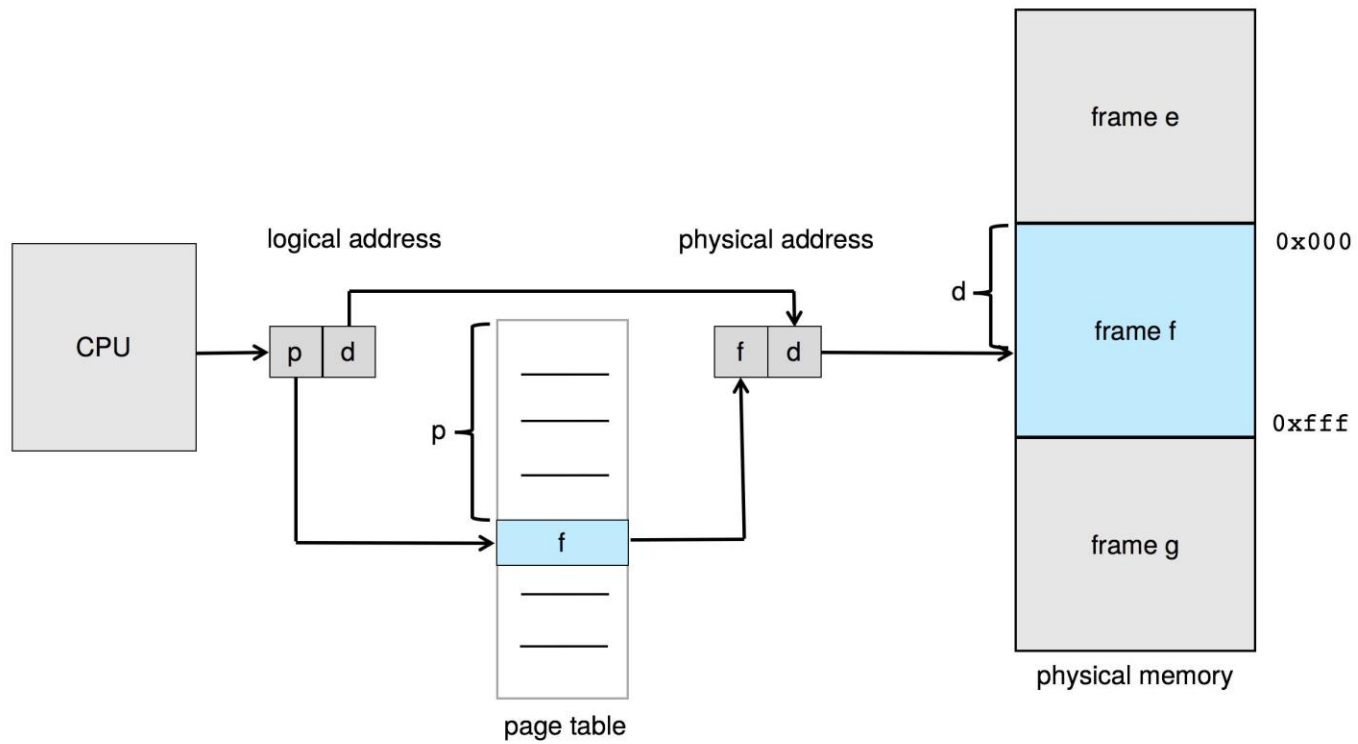
2. Page Offset(d): It tells the exact word on that page which the CPU wants to read. It requires no translation as the page size is the same as the frame size so the place of the word which CPU wants access will not change.

Page table: A page table contains the frame number corresponding to the page number of some specific process. So, each process will have its own page table. A register called Page Table Base Register (PTBR) which holds the base value of the page table.

Paging Model of Logical and Physical Memory



Paging Hardware



Paging Example

Logical address: $n = 2$ and $m = 4$. Using a page size of 4 bytes and a physical memory of 32 bytes (8 pages)

0	a
1	b
2	c
3	d
4	e
5	f
6	g
7	h
8	i
9	j
10	k
11	l
12	m
13	n
14	o
15	p

logical memory

0	5
1	6
2	1
3	2

page table

0	
4	i j k l
8	m n o p
12	
16	
20	a b c d
24	e f g h
28	

physical memory

$$\text{Physical Address} = (\text{Frame Number} \times \text{Frame Size}) + \text{Offset}$$

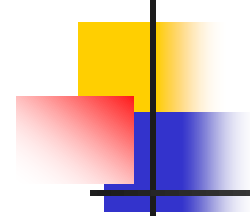


Advantages of Paging:

1. There is **no external fragmentation** as it allows us to store the data in a non-contiguous way.
2. Swapping is easy between equal-sized pages and frames.

Disadvantages of Paging:

1. As the size of the frame is fixed, so it may **suffer from internal fragmentation.**
2. The access time increases because of paging as the main memory has to be now accessed two times.
3. For every process, we have an independent page table and maintaining the page table is **extra overhead.**

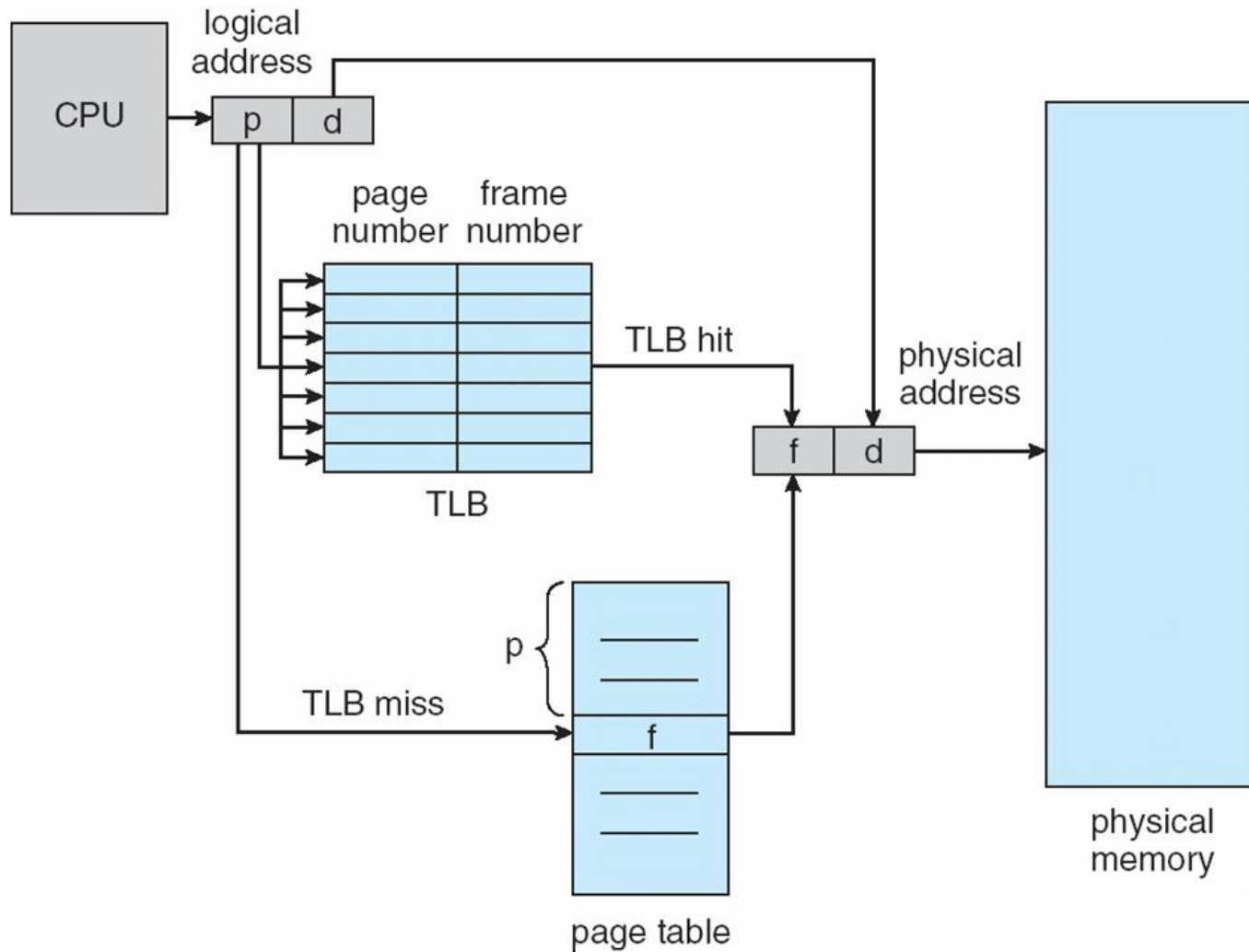


Paging with Translation Look-aside Buffers (TLBs)

- Since Page table is kept in main memory it can be accessed by the pointer **Page-table base register** (PTBR)
- **Page-table length register** (PTLR) indicates size of the page table
- In this scheme every data/instruction access requires two memory accesses
 - One for the page table and one for the data / instruction
- The two memory access problem can be solved by the use of a special fast-lookup hardware cache called **associative memory or translation look-aside buffers (TLBs)**
- Each TLB entry – uniquely identifies each process to provide address-space protection for that process.
- TLBs are typically small size compared to Page Table.

- 
- Each time when address translation need to be done, the page reference will be checked in the TLB first.
 - If it present then it is called **TLB Hit**, then its frame number will be passed. If it is not present in the table then it called **TLB Miss**.
 - On a TLB miss, then page will be searched in the Page Table and later value is loaded into the TLB for faster access next time.
 - Paging hardware with TLB consists of both TLB table and Page Table.

Paging Hardware With TLB





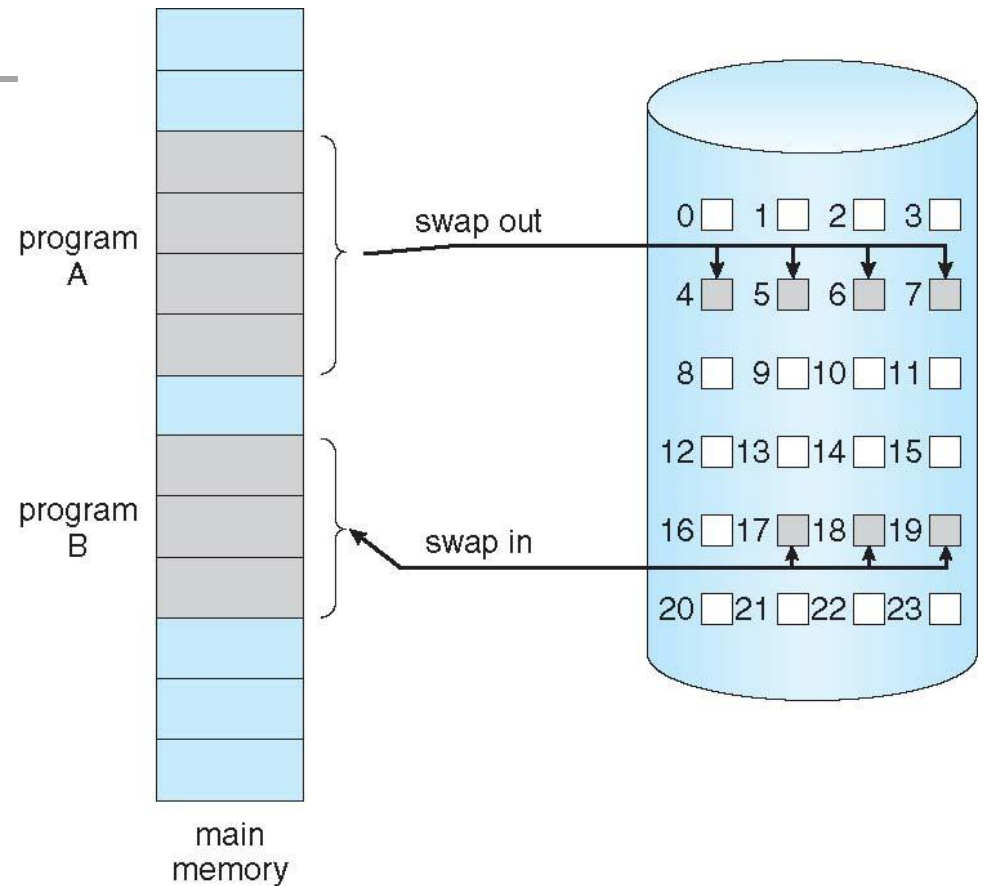
❑ Virtual Memory

- Virtual Memory is a storage mechanism which offers user an illusion of having a very big main memory.
- It is done by treating a part of secondary memory as the main memory.
- In Virtual memory, the user can store processes with a bigger size than the available main memory.
- Instead of loading one long process in the main memory, the OS loads the various parts of more than one process in the main memory.
- Virtual memory is mostly implemented with demand paging and demand segmentation.



□ Demand Paging

- Demand paging is a memory management technique used by operating systems to optimize the use of memory resources.
- In demand paging, we only load the required pages of a process into the main memory instead of loading the entire process.
- A demand paging mechanism is very much similar to a paging system with swapping where processes stored in the secondary memory and pages are loaded only on demand, not in advance.
- During the program execution, if the program references a page that may not be available in the main memory then this scheme will bring that page on demand from the secondary storage.



- It uses a **Lazy swapper** – never swaps a page into memory unless page will be needed.
- Swapper that deals with pages is called **Pager**.



Advantages of Demand Paging:

- Less I/O needed, no unnecessary I/O
- Less memory needed
- Faster response
- More users can be use memory at the same time.



Page Fault

If there is a reference to a page, first reference to that page will trap to operating system:

Steps to follow during page fault:

1. Operating system looks at another table to decide:

Invalid reference $\Rightarrow$ abort

Just not in memory

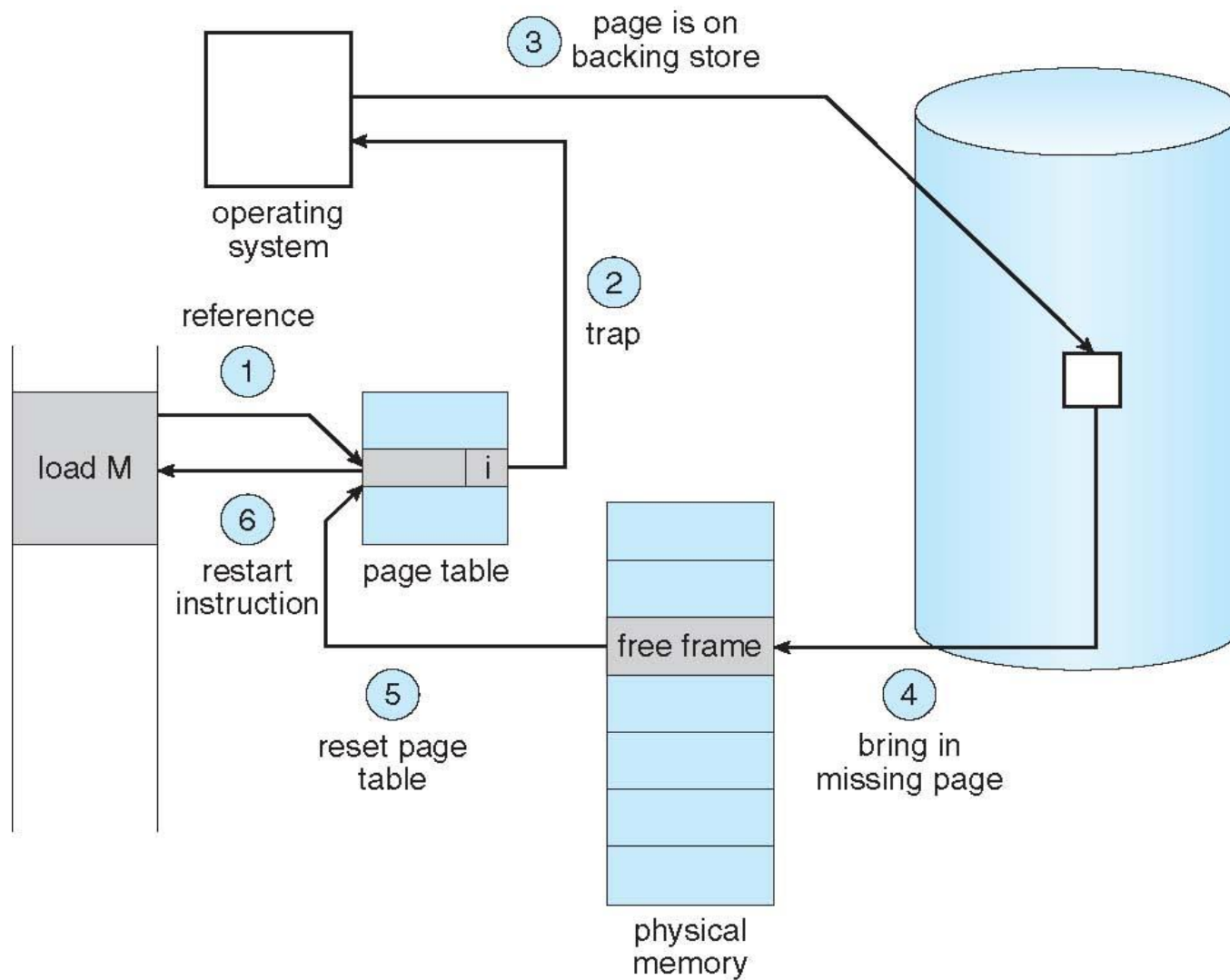
2. Find free frame

3. Swap page into frame via scheduled disk operation

4. Reset tables to indicate page now in memory

Set validation bit = v

5. Restart the instruction that caused the page fault





Thrashing

Thrashing is a condition or a situation where the operating system is spending a major portion of its time in servicing the page faults, rather than actual processing.

The effect of Thrashing is:

- CPU becomes idle.
- Decreasing the utilization increases the degree of multiprogramming and hence bringing more processes at a time which in fact increases the thrashing exponentially.



❑ Page Replacement Algorithm

- Page Replacement Algorithm decides which page to remove, also called swap out when a new page needs to be loaded into the main memory.
- Page Replacement happens when a requested page is not present in the main memory and the available space is not sufficient for allocation to the requested page.
- A page replacement algorithm tries to select which pages should be replaced so as to minimize the total number of page misses.
- If a process requests for page and that page is found in the main memory then it is called **page hit**.
- If the requested page is not found in the main memory then it is called as **page miss** or **page fault**.



Some Page Replacement Algorithms :

- First In First Out (FIFO)
- Least Recently Used (LRU)
- Optimal Page Replacement



First In First Out (FIFO)

- This is the simplest page replacement algorithm. In this algorithm, the OS maintains a queue that keeps track of all the pages in memory, with the oldest page at the front and the most recent page at the back.
- When there is a need for page replacement, the FIFO algorithm, swaps out the page at the front of the queue, that is the page which has been in the memory for the longest time.

Advantages

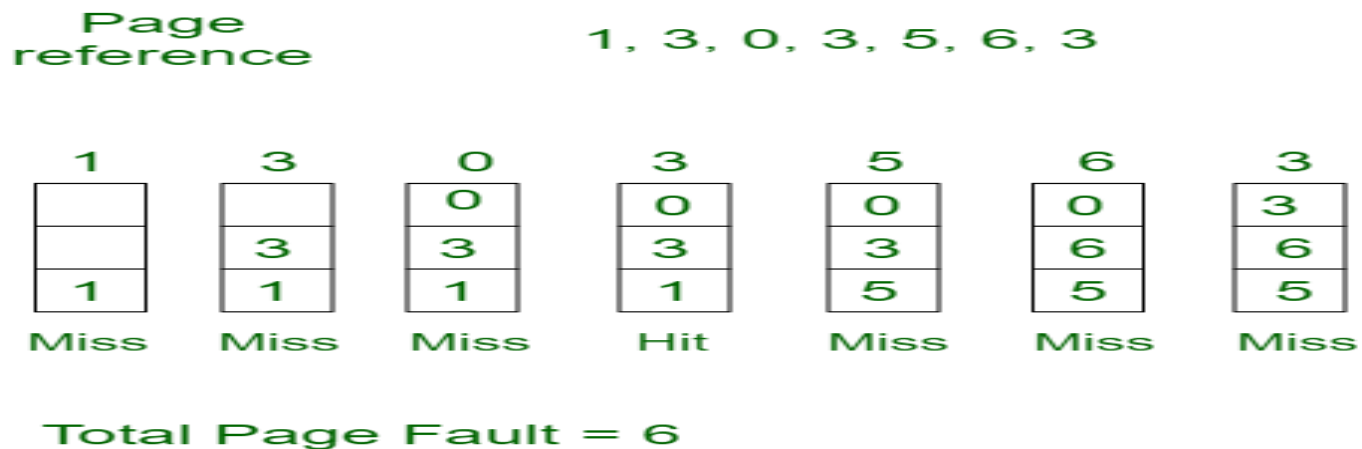
- Simple and easy to implement.
- Low overhead.

Disadvantages

- Poor performance.
- Suffers from Belady's Anomaly(i.e. more page faults when we increase the number of page frames).

For Example:

Consider page reference string 1, 3, 0, 3, 5, 6 with 3 page frames. Find number of page faults.



Belady's anomaly – Belady's anomaly proves that it is possible to have more page faults when increasing the number of page frames while using the First in First Out (FIFO) page replacement algorithm.

Optimal Algorithm

- Replace page that will not be used for longest period of time
- OS needs future knowledge of page references in advance.
- Need to sort according to their next usage/ reference
- Overhead for OS & complex to implement.

reference string

7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1

7	7	7	2		2	2		2		2							7		
	0	0	0		0	4		0		0							0		
		1	1		3	3		3		1							1		

page frames

Least Recently Used (LRU) Algorithm

- Use past knowledge rather than future
- Replace page that has not been used in the most amount of time
- Associate time of last use with each page.

reference string

7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1

7	7	7	2		2		4	4	4	0			1		1		1		
	0	0	0		0		0	0	3	3			3		0		0		
		1	1		3		3	2	2	2			2		2		7		

page frames

- 12 faults – better than FIFO but worse than OPT

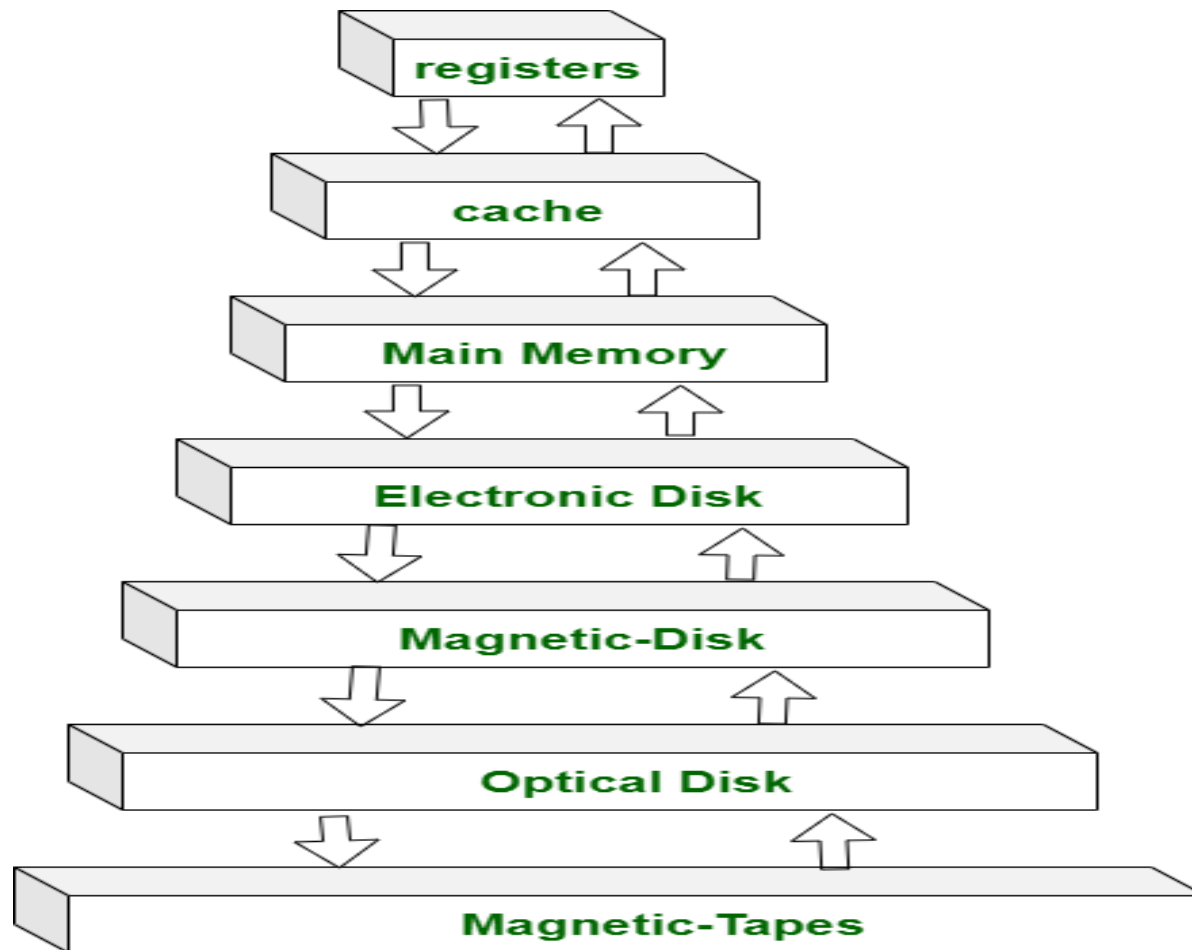
- 
-
- **Storage Management:**
 - **Overview of Mass Storage Structure**
 - **Disk Structure**
 - **Hard Disk Drive(HDD) Scheduling**



Overview of Mass Storage Structure

- *Mass Storage* : persistent storage; large volume of data can be stored permanently (powering off will not cause loss of data)
 - **Secondary storage**: always online; hard disk
 - **Tertiary storage**; tapes, etc.

Overview of Mass Storage Structure



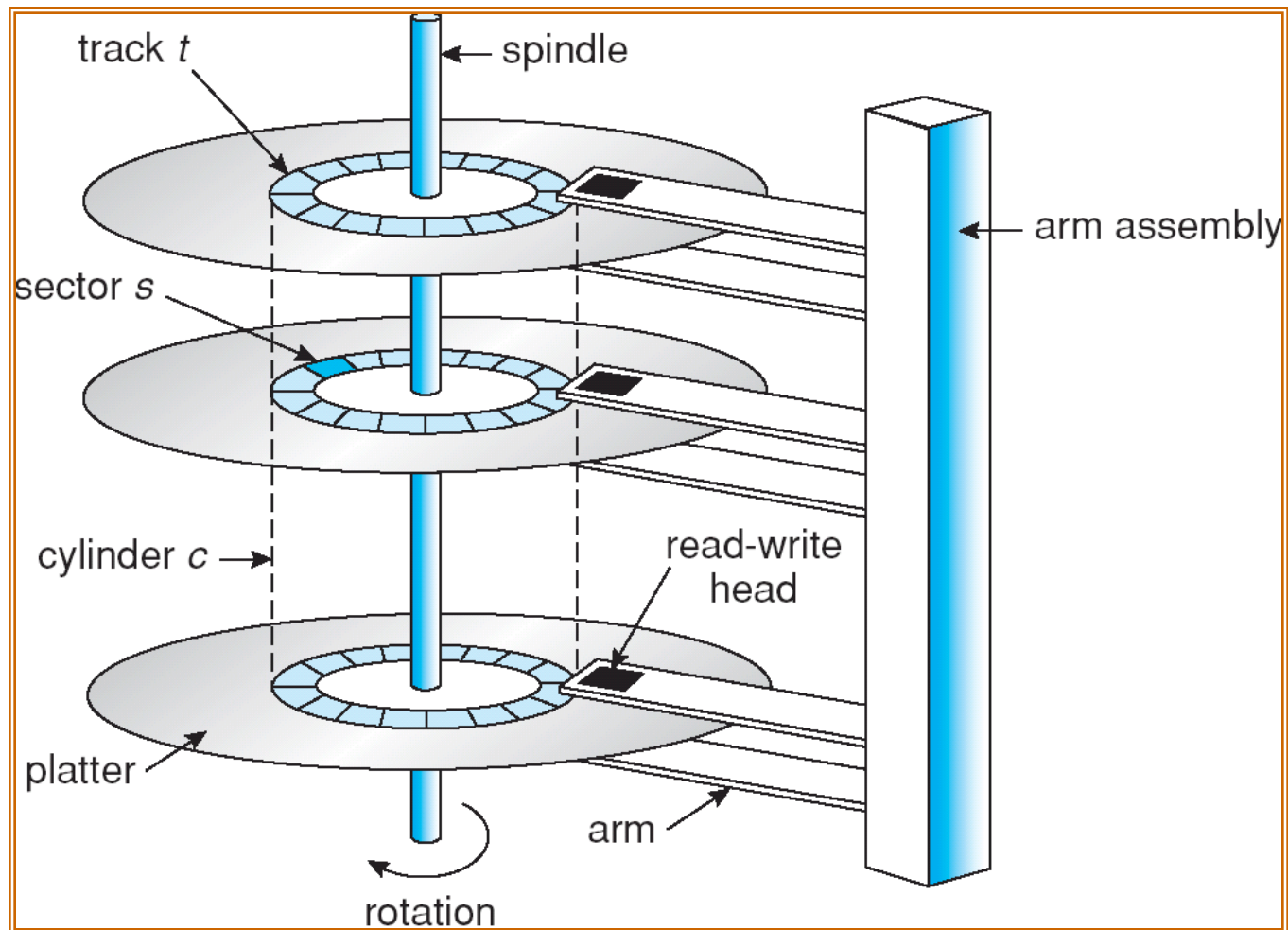
Storage Device Hierarchy.



Magnetic Disks

- Magnetic disks provide bulk of secondary storage of modern computers
 - Drives rotate at 60 to 200 times per second
 - **Transfer rate** is rate at which data flow between drive and computer
 - **Positioning time** (random-access time) is time to move disk arm to desired cylinder (**seek time**) and time for desired sector to rotate under the disk head (**rotational latency**)
 - Head crash results from disk head making contact with the disk surface
 - That's bad
 - Disks can be removable

Moving-head Disk Mechanism

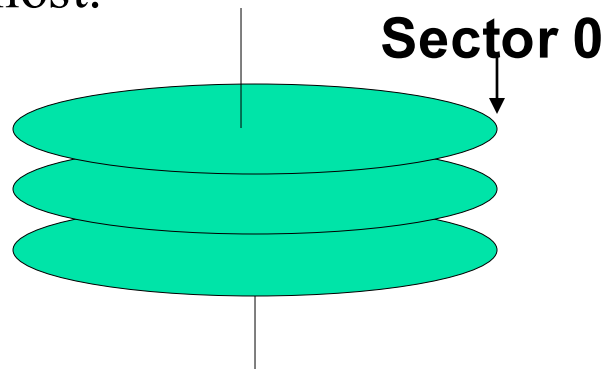


- Magnetic tape

- Was early secondary-storage medium
- Relatively permanent and holds large quantities of data
 - 20-200GB typical storage
- Mainly used for backup, storage of infrequently-used data, transfer medium between systems
- Access time slow
- Random access ~1000 times slower than disk
- Once data under head, transfer rates comparable to disk
- Common technologies are 4mm, 8mm, 19mm, LTO-2 and SDLT

Disk Structure

- A disk drive is addressed as a large **1-dimensional array of blocks**, where the logical **block** is the smallest **unit of transfer**.
- The 1-dimensional array of **blocks is mapped into the sectors** of the disk **sequentially**.
 - Sector 0 is the first sector of the first track on the outermost cylinder.
 - Mapping proceeds in order through that track, then the rest of the tracks in that cylinder, and then through the rest of the cylinders from outermost to innermost.





Disk Scheduling

- The operating system is responsible for using hardware efficiently — for the disk drives, this means having a fast **access time** and large **disk bandwidth**.
- **Disk access time** has two major components
 - *Seek time* is the time for the disk to move the head to the cylinder containing the desired sector (block).
 - *Rotational latency* is the additional waiting time for the disk to rotate the desired sector under the disk head.
- **Minimize seek time**
- Seek time $\approx$ **seek distance** (between cylinders)
- Disk bandwidth is the total number of bytes transferred, divided by the total time between the first request for service and the completion of the last transfer.



Disk Scheduling

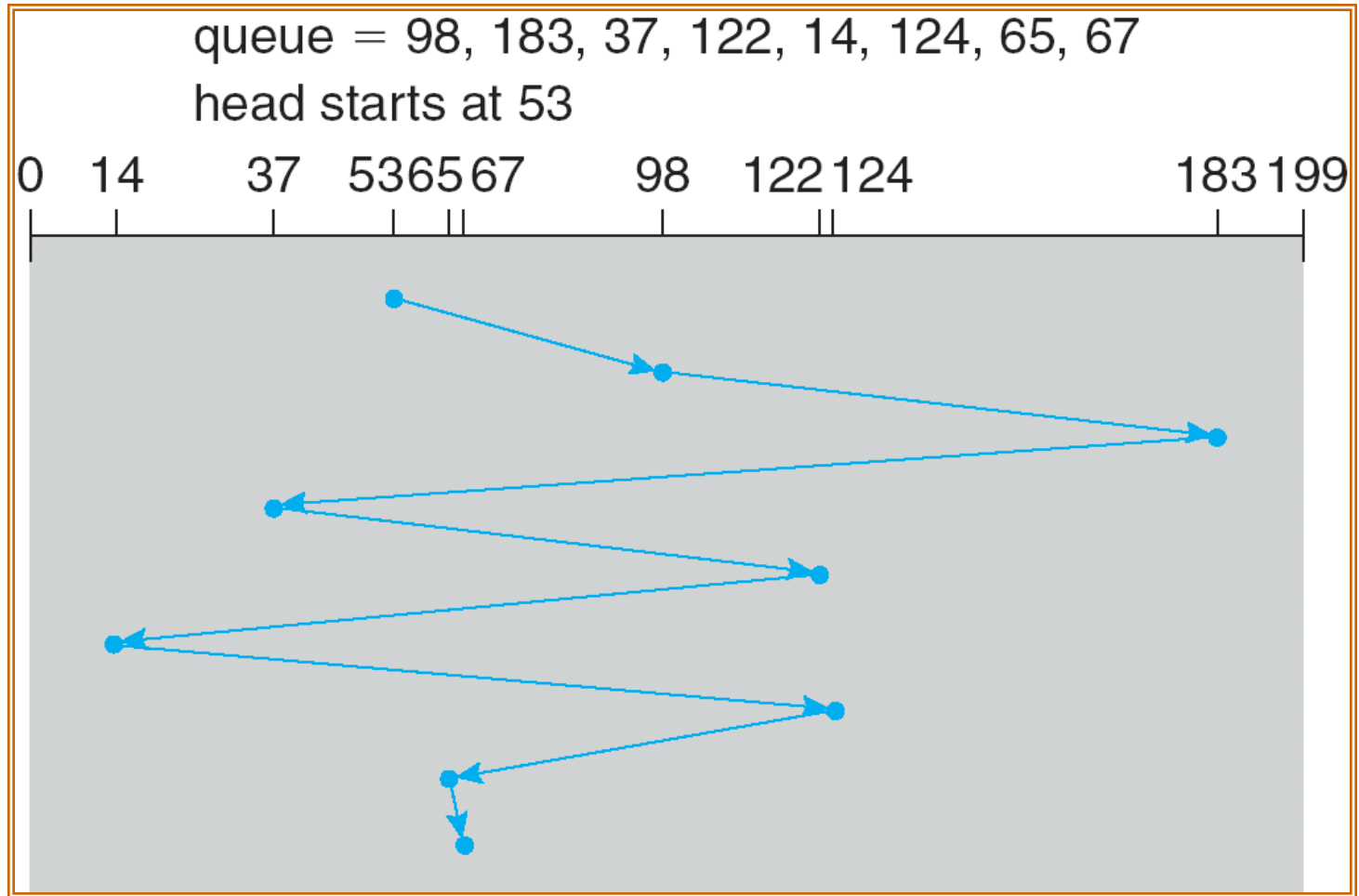
- Several algorithms exist to **schedule** the servicing of **disk I/O requests**.
- Assume disk has cylinders from 0 to 199.
- We illustrate them with a *request queue*. In the queue we have requests for blocks sitting in various cylinders. We just focus on the **cylinder numbers**.

98, 183, 37, 122, 14, 124, 65, 67 (these are cylinder numbers)

Head pointer: 53 (the head is currently on cylinder 53)

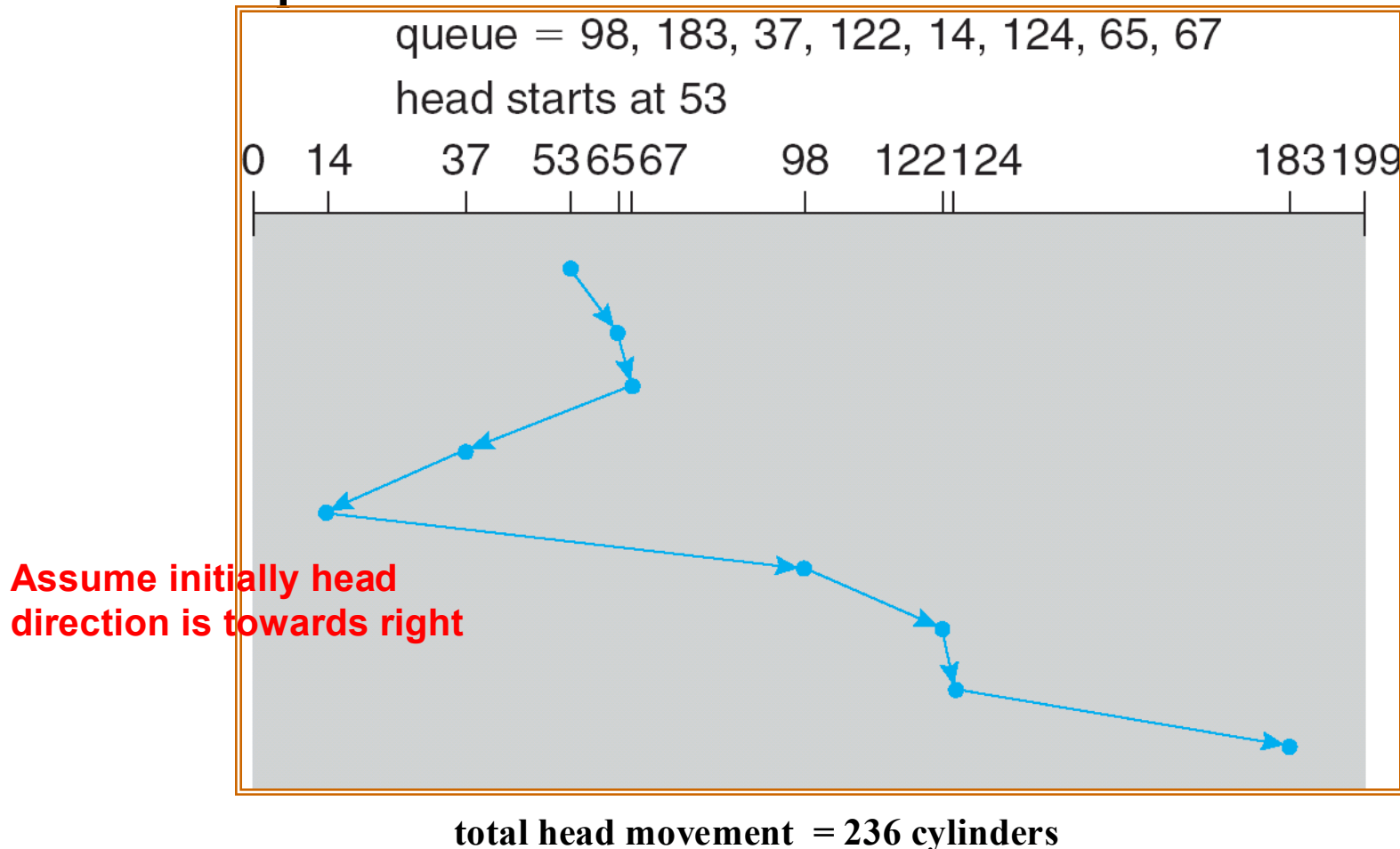
We have 8 requests queued. They are for blocks sitting on cylinders
98, 183, ...

FCFS Algorithm



Disk Scheduling: SSTF Algorithm

- Selects the request with the *minimum seek time* from the current head position.
- SSTF scheduling is a form of SJF scheduling; may cause starvation of some requests.

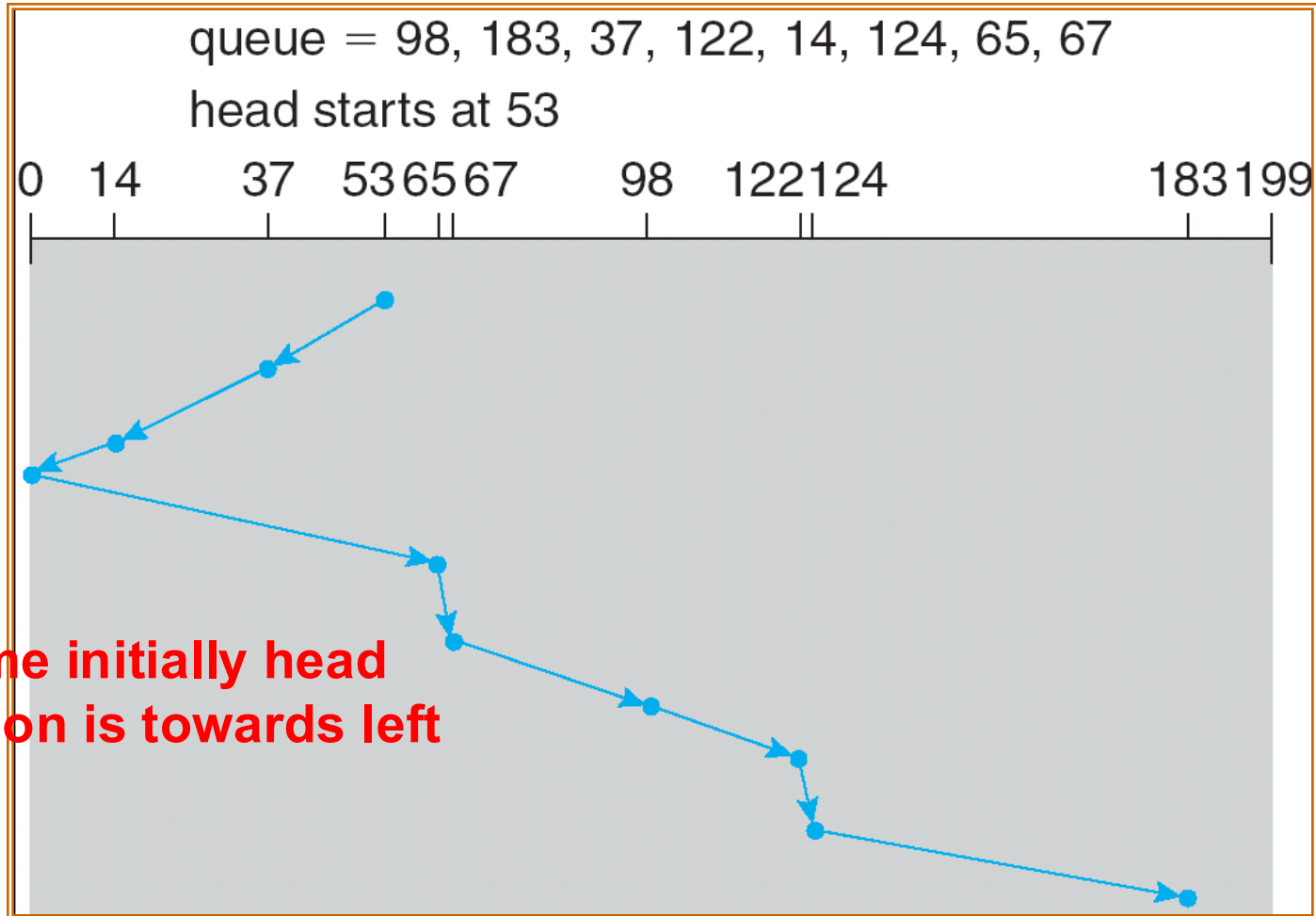




Disk Scheduling: SCAN/ELEVATOR Algorithm

- The disk arm starts at one end of the disk, and moves toward the other end, servicing requests until it gets to the other end of the disk, where the head movement is reversed and servicing continues.
- Also called the *Elevator algorithm*.
- Several variations of the algorithm exist:
 - C-SCAN
 - LOOK
 - C-LOOK

Disk Scheduling: SCAN



total head movement = 236 cylinders



Disk Scheduling: C-SCAN

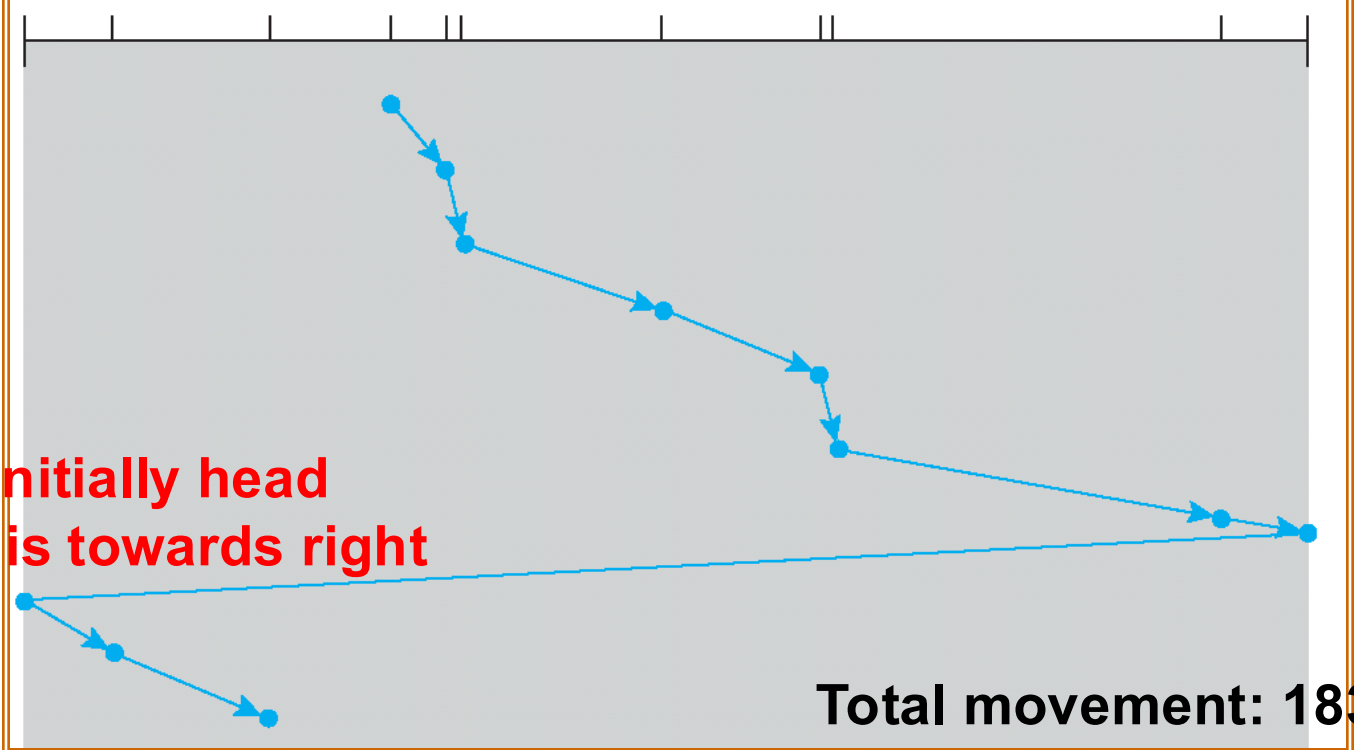
- C-SCAN: Circular SCAN
- Provides a *more uniform wait time* than SCAN.
 - Wait time for a request: time between arrival of a request to the queue and the completion of handling the request.
- The head moves from one end of the disk to the other; servicing requests as it goes. When it reaches the other end, however, it immediately returns to the beginning of the disk, without servicing any requests on the return trip.
- Treats the cylinders as a **circular list** that wraps around from the last cylinder to the first one.

Disk Scheduling

queue = 98, 183, 37, 122, 14, 124, 65, 67

head starts at 53

0 14 37 53 65 67 98 122 124 183 199

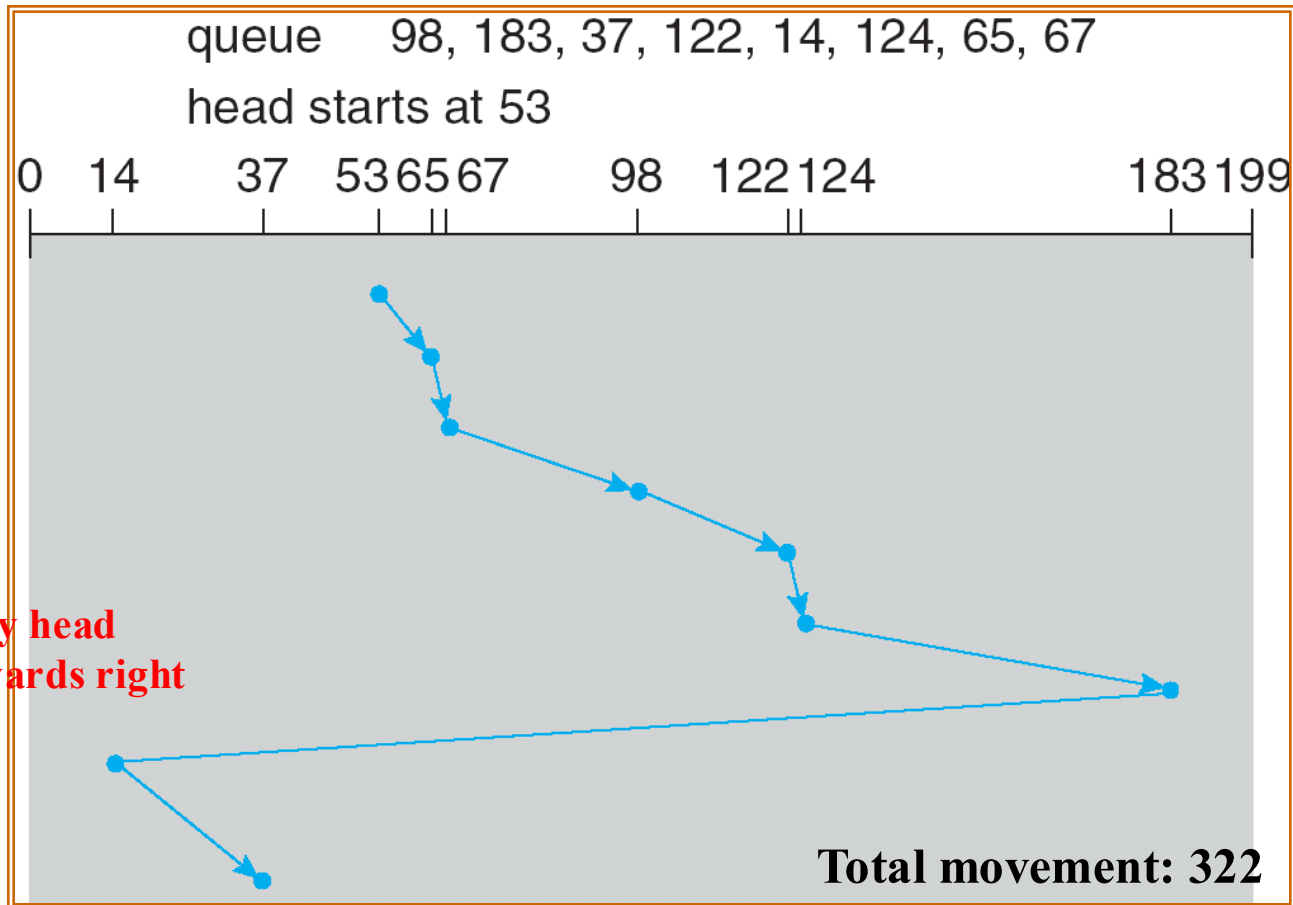


Assume initially head direction is towards right

Total movement: 183

Disk Scheduling: C-LOOK

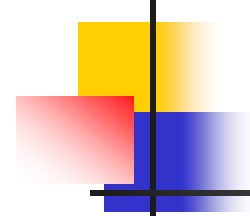
- Version of C-SCAN
- Arm only goes as far as the **last request** in each direction, then reverses direction immediately (without first going all the way to the end of the disk); and goes to the first request in the other end of the disk.





Disk Scheduling: LOOK

- From 53 to 183 (sweep meanwhile)
- From 183 to 14 (sweep meanwhile)
- Total = 299



Selecting a Disk-Scheduling Algorithm

- SSTF is common and has a natural appeal
- SCAN and C-SCAN perform better for systems that place a **heavy load** on the disk.
- Performance depends on the **number and types of requests**.
- Requests for disk service can be influenced by the file-allocation method.
- The disk-scheduling algorithm should be written **as a separate module** of the operating system, allowing it to be replaced with a different algorithm if necessary.
- Either SSTF or LOOK is a reasonable choice for the default algorithm.

